

Utvärdering av ROS2 som styrsystem för LIMB-projektet

13 augusti 2026

Uppgift

En utvärdering ska genomföras av möjligheten att implementera ROS2 i systemet, inklusive för- och nackdelar.

1 Bakgrund och nulägesbeskrivning

LIMB är ett studentprojekt vid Mälardalens universitet som syftar till att utveckla en bionisk arm för användning i rehabilitering av strokepatienter. Systemet som utvärderas i detta dokument styr en torso med vänsterarm via EMG-signaler från en egenbyggd sensor.

1.1 Nuvarande arkitektur

Systemet är idag uppbyggt utan ROS2 enligt följande struktur:

- **Mikrokontroller-lager:** ESP32-C3-Zero-moduler sitter monterade vid respektive motor/led. De styr motorerna och läser av lokala sensorer, och kommunicerar över CAN-bussen.
- **Huvuddator:** En NVIDIA Jetson AGX Orin kör huvudprogrammet. Detta program:
 - läser sensorvärden över CAN från ESP32-modulerna,
 - läser EMG-/användardata över Bluetooth,
 - beräknar hur armen ska bete sig utifrån denna information,
 - skickar ut nya ledvinklar över CAN till ESP32-modulerna.
- **Programmeringsspråk:** Styrlogiken på Jetson-datorn är skriven i Python, i ett i huvudsak monolitiskt program.

2 Vad är ROS2?

ROS2 (Robot Operating System 2) är ett ramverk för att bygga robotmjukvara, uppbyggt kring ett antal grundbegrepp:

- **Nodes:** fristående processer som var och en sköter en avgränsad uppgift, t.ex. sensorinläsning eller styrlogik. Kan skrivas i Python (`rclpy`) eller C++ (`rclcpp`).
- **Topics:** namngivna kanaler där noder publicerar eller prenumererar på data, utan att känna till varandra direkt (publish/subscribe-mönster).
- **Messages:** strikt typade datastrukturer som skickas på topics.

- **Services och actions:** synkrona fråga/svar-anrop respektive längre uppgifter med statusuppdateringar, för sådant som inte passar publish/subscribe.
- **Paket och workspace:** kodorganisation där flera paket byggs tillsammans med byggverktyget `colcon`.
- **Launch-filer:** startar flera noder samtidigt med rätt konfiguration.

Kommunikationen mellan noder sker via DDS (Data Distribution Service), vilket ger viss overhead jämfört med direktkommunikation men även fördelar såsom decentraliserad upptäckt av noder och konfigurerbar tillförlitlighet (QoS).

3 Fördelar med att införa ROS2

- **Modularitet:** Dagens monolitiska Python-program skulle kunna delas upp i separata noder t.ex. en CAN-nod, en EMG/BLE-nod och en beslutsnod med tydliga gränssnitt mellan dem. Detta gör det enklare att byta ut eller vidareutveckla en del (t.ex. EMG-klassificeringen) utan att röra resten av systemet.
- **Färdiga standardpaket:**
 - `ros2_socketcan` kan ersätta egenskriven CAN-hantering.
 - `ros2_control` ger färdig infrastruktur för ledstyrning, inklusive tillståndshantering som är relevant ur säkerhetssynpunkt eftersom armen är avsedd att användas av patienter.
 - `tf2` hanterar koordinatramar mellan torso och arm, vilket är användbart om kinematik ska räknas ut mer systematiskt.
 - `RViz2` ger visualisering av sensordata och ledvinklar för felsökning.
 - `Gazebo` möjliggör simulering av armen innan ändringar testas på fysisk hårdvara.
- **Loggning och reproducerbarhet:** `ros2 bag` gör det möjligt att spela in och spela upp sensordata/kommandon, vilket underlättar felsökning och utvärdering av EMG-tolkningen.
- **Överlämning mellan studentgrupper:** Eftersom LIMB är ett återkommande studentprojekt med nya deltagare varje läsår, gör en standardiserad nod-/topic-struktur det enklare för nästa grupp att sätta sig in i och vidareutveckla systemet jämfört med ett enskilt monolitiskt skript.
- **Framtida integration:** Om projektet i framtiden ska kombinera EMG-styrning med andra sensortyper (t.ex. kamerabaserad armvinkel-spårning) blir det enklare att slå ihop datakällor via topics än att hårdkoda ihop separata program.

4 Nackdelar och risker med att införa ROS2

- **Latens:** EMG-styrning kräver låg och jämn latens mellan signal och motorrespons. DDS-lagret i ROS2 lägger till overhead jämfört med en direkt seriell eller BLE-koppling. Det går att justera QoS-profiler och byta RMW-implementation (t.ex. CycloneDDS eller Zenoh), men detta måste testas och verifieras mot den befintliga lösningen.
- **Inlärningskurva:** Om ingen i projektgruppen har tidigare erfarenhet av ROS2 så tillkommer nya begrepp (noder, topics, paket, launch-filer, `colcon`-byggsystem) som tar

tid att lära sig, vilket kan tränga undan tid för det faktiska utvecklingsarbetet under projektets gång.

- **Ökad komplexitet för en i grunden enkel pipeline:** Om systemets pipeline (EMG-signal → tolkning → motorstyrning) förblir relativt linjär och enkel, riskerar ROS2 att införa mer strukturell komplexitet än den faktiska nyttan motiverar.
- **Omskrivningskostnad:** Den befintliga koden i `src/layers` fungerar redan. En övergång till ROS2 innebär att fungerande kod måste skrivas om och testas på nytt, med risk för att införa nya buggar i ett system som styr fysisk hårdvara.
- **Plattformsberoenden:** Vilken ROS2-distribution som går att installera rakt av beror på vilken Ubuntu-version som körs på Jetson AGX Orin (styrd av JetPack-versionen). Detta kan kräva extra arbete i form av Docker-containrar eller kompilering från källkod.

5 Rekommenderat tillvägagångssätt

1. Kartlägg den faktiska fördelningen mellan ren in-/utdatahantering och egentlig beslutslogik i `src/layers`, för att bedöma hur mycket som skulle vinna på att brytas ut i separata noder.
2. Mät den nuvarande end-to-end-latensen (från EMG-signal till utskickad ledvinkel på CAN-bussen) i det befintliga systemet.
3. Bygg ett minimalt ROS2-prototypsystem (en publisher- och en subscriber-nod) och mät motsvarande latens under samma förhållanden, för att få ett konkret jämförelsetal.

6 Slutsats

ROS2 skulle kunna ge LIMB-projektet en mer modulär, lättöverlämnad och utbyggbar arkitektur, med tillgång till färdiga standardpaket för CAN-kommunikation, ledstyrning och simulering. Samtidigt medför bytet en inlärningskostnad, risk för ökad latens i en tidskritisk EMG-styrslina, och en omskrivning av kod som redan fungerar. Rekommendationen är att genomföra en liten, mätbar prototyp innan en fullständig konvertering påbörjas, istället för att fatta beslutet enbart utifrån teoretiska för- och nackdelar.